



2-if

1. 简介

本节主旨：本节讲解选择结构，包括 `if/else` 语句、`switch/case` 语句与逻辑表达式；介绍现代 C++ 中带初始化器的 `if` 与 `switch` 语句；同时，介绍变量作用域的概念；最后简要引入 `[[fallthrough]]`、`[[likely]]` / `[[unlikely]]` 等 C++ 属性（attributes）。

学习目标：学完本节后，你应该能够：

- 使用 `if`、`if-else` 与嵌套 `if-else` 表达选择结构，理解布尔变量与标志变量的用法；
- 构造逻辑表达式（`&&`、`||`、`!`），理解短路求值与运算符优先级；
- 使用 `switch/case` 表达“离散常量”的多分支选择，理解 `case`、`break`、`default` 与 `fallthrough`；
- 掌握 `if/switch` 语句带初始化器与变量作用域。

本节重点：

- `if-else` 与嵌套 `if-else`、`else if` 链；
- `switch/case` 的语法与典型用法，`[[fallthrough]]` 的用法；
- `if/switch` 语句带初始化器与变量作用域。

本节难点：

- 嵌套 `if-else` 的逻辑梳理（借助流程图）；
- `if/switch` 语句带初始化器与变量作用域。

2. 再谈“程序 = 数据结构 + 算法”

仅有顺序语句的程序功能是非常受限的。现实世界无时无刻不充满着选择，为了建模这种情境，程序设计语言引入了分支/选择结构，包括 `if` 语句与 `switch` 语句。它们根据给定条件（condition）是否成立，决定要采取的动作。这里涉及条件判断，条件判断的结果只有两种可能性，要么为真，要么为假，属于逻辑（logic）的范畴，因此用于表示条件的表达式称为逻辑表达式（logical expressions）。本节即介绍分支/选择结构以及它们所依赖的逻辑表达式。

3. min

3.1 min-of-two

求两个整数的最小值。

```
min-of-two.cpp

1  // File: min-of-two.cpp
2
3  #include <format>
4  #include <iostream>
5
6  int main() {
7      int a{};
8      int b{};
9
10     std::cin >> a >> b;
11
12     int min{};
13     if (a >= b) {
14         min = b;
15     } else {
16         min = a;
17     }
18
19     std::cout << std::format("min({}, {}) = {}\n", a, b, min);
20
21     return 0;
22 }
```

样例输入

1 3 5

```
int min(3, 5) = 3
```

Line 13 到 Line 17 是表示选择的 `if-else` 语句，它首先判断 `a >= b` 是否成立，如果成立，则意味着最小值为 `b` (`min = b;`)，否则意味着最小值为 `a` (`min = a;`)。 `if-else` 语句包含 `if` 子句与 `else` 子句，分别放在一对花括号中。注意， `if-else` 语句不以 `;` 结尾。

这里有一个重要的细节，需要强调：变量 `min` 的定义出现在第 12 行，而不是更早的地方，比如第 8 行之后。这符合 C++ Core Guidelines 中的如下基本规范：

 C++ Core Guidelines [ES.22: When you need a variable, prefer to declare it at its first use](#): 直到有值可初始化、且确需使用时才声明变量。

更本质的原因是，

 C++ Core Guidelines [ES.5: Keep scopes small](#): 保持作用域尽可能小。

3.1.1 `a >= b`

`a >= b` 中的 `>=` 是关系 (relational) 运算符，其它关系运算符包括 `>`、`<`、`<=`。

关系运算符与相等性 (equality) 运算符 (包括 `==` 与 `!=`) 同属 [比较 \(comparison\) 运算符](#)。比较运算符作用在左右两边的操作数上，其结果的类型为 `bool`：如果条件成立，则为 `true`；否则为 `false`。

3.1.2 `? :`

C++ 提供了唯一的三目运算符 (ternary operator) `? :` (也称条件运算符, conditional operator, 参见 [operator_other @ cppreference](#))，由此构成的表达式也可以表达 `if-else` 的含义。使用 `? :`，可以如下计算两个整数之间的最小值：

代码块

```
1 int min{a < b ? a : b};
```

这行代码等价于以下代码：

代码块

```
1 int min{};
2 if (a < b) {
3     min = a;
4 } else {
```

```
5     min = b;
6 }
```

由此可知，表达式 `C ? A : B` 整体的值取决于条件 `C` 是否成立，如果 `C` 成立，则为表达式 `A` 的值，否则为表达式 `B` 的值。根据 C++ 标准的规定，`A` 与 `B` 永远不会同时被求值（只会求值其中一个）。

! 适当使用 `?:` 可以使代码简洁，但是过度使用会破坏代码的可读性。

3.2 min-of-three

求三个整数的最小值。

min-of-three.cpp

```
1 // File: min-of-three.cpp
2
3 #include <format>
4 #include <iostream>
5
6 int main() {
7     int a{};
8     int b{};
9     int c{};
10
11     std::cin >> a >> b >> c;
12
13     int min{};
14     if (a > b) {
15         if (b > c) {
16             min = c;
17         } else { // b <= c and b < a
18             min = b;
19         }
20     } else { // a <= b
21         if (a < c) {
22             min = a;
23         } else { // c <= a <= b
24             min = c;
25         }
26     }
27
28     std::cout << std::format("min({}, {}, {}) = {}\n", a, b, c, min);
29
30     return 0;
}
```

```
31 }
```

样例输入

```
1 3 2 5
```

预期输出

```
1 min(3, 2, 5) = 2
```

本例演示了嵌套 `if-else` 的用法，没有引入其它新的知识点。

4. 带初始化器的 `if` 与变量作用域

输入年龄，判断是否成年（18 岁），并输出距离成年或已成年多少年；若已成年，提示拥有选举权和被选举权。



《中华人民共和国宪法》第三十四条规定，中华人民共和国年满十八周岁的公民，不分民族、种族、性别、职业、家庭出身、宗教信仰、教育程度、财产状况、居住期限，都有选举权和被选举权；但是依照法律被剥夺政治权利的人除外。

本节通过三种写法的对比，理解变量作用域（scope）与生命周期（lifetime）的概念，掌握 C++17 引入的 `if` 带初始化器语法，体会它如何同时解决“变量泄漏”与“分支内不可见”两个问题。

4.1 `adult-outside`

写法一：变量声明在 `if` 外部（传统写法）

adult-outside.cpp

```
1 // File: declare-outside.cpp
2 // Way 1: years is declared outside the if statement.
3 // Its scope covers the rest of main(), which may cause misuse.
4
5 #include <format>
6 #include <iostream>
7
8 int main() {
9     int age{};
10    std::cin >> age;
11
```

```

12     int years = age - 18; // years means age - 18, not the age itself
13     if (years >= 0) {
14         std::cout << std::format("adult ({} year(s))\n", years);
15     } else {
16         std::cout << std::format("adult in {} year(s)\n", -years);
17     }
18
19     // A later check intends to test whether the age reaches 60,
20     // but years is misused as if it were the age.
21     if (years >= 60) {
22         std::cout << std::format("senior\n");
23     }
24
25     return 0;
26 }

```

样例输入 / 预期输出

```

1  25 =>  adult (7 year(s))
2  17 =>  adult in 1 year(s)
3  70 =>  adult (52 year(s))

```

问题分析：

- `years` 的实际含义是 `age - 18`，不是年龄，但这个名字很容易让人误以为它是年龄；
- 后续代码 `if (years >= 60)` 本意是判断年龄是否达到 60，却错误地使用了 `years`；
- 假设输入 `age = 70`，则 `years = 52`，`52 >= 60` 为假，不会输出 `senior`，但 70 岁显然已进入老年——这种逻辑错误在编译期无法发现，运行时才暴露；
- 根本原因是 `years` 的作用域过大，泄漏到了后续代码中。

4.2 adult-inside

写法二：变量声明在 if 分支内部

adult-inside.cpp

```

1  // File: declare-inside.cpp
2  // Way 2: years is declared inside each branch of the if statement.
3  // No leakage, but the two branches cannot share one variable.
4
5  #include <format>
6  #include <iostream>
7
8  int main() {

```

```

9    int age{};
10   std::cin >> age;
11
12   if (age >= 18) {
13       int years = age - 18; // visible only in this branch
14       std::cout << std::format("adult ({} year(s))\n", years);
15   } else {
16       int years = age - 18; // not visible here, so it is recomputed
17       std::cout << std::format("adult in {} year(s)\n", -years);
18   }
19
20   // years is not visible here any more; the following line
21   // would not compile:
22   // if (years >= 60) { ... } // error: 'years' was not declared
23
24   return 0;
25 }

```

样例输入 / 预期输出

```

1  25 =>  adult (7 year(s))
2  17 =>  adult in 1 year(s)

```

在 `adult-inside` 中，两个 `years` 的作用域是它们各自所在的 `if` 或 `else` 分支，不会泄漏到外部。然而，这也使得 `if` 和 `else` 分支无法共享同一个 `years` 变量，导致第 13 行与第 16 行代码重复。代码重复（复制、粘贴）是 bugs 的温床，也会带来维护负担。

4.3 `adult-if-init`

写法三：if 带初始化器（C++17）

语法（参见 [cppreference: if 语句](#)；该特性由 ISO C++ 提案 [P0305R1](#) 引入）：

语法

```

1  if (init-statement; condition) {
2      // 语句块
3  } else {
4      // 语句块
5  }

```

初始化语句中声明的变量，作用域覆盖整个 `if-else` 结构，但出了这个结构就立即消失。完整示例：

```

1 · ··File: if-init.cpp
2  // Way 3: if statement with initializer (C++17).
3  // The scope of years covers the whole if-else statement,
4  // and ends right after it.
5
6  #include <format>
7  #include <iostream>
8
9  int main() {
10     int age{};
11     std::cin >> age;
12
13     if (int years = age - 18; years >= 0) {
14         std::cout << std::format("adult ({} year(s))\n", years);
15     } else {
16         std::cout << std::format("adult in {} year(s)\n", -years);
17     }
18
19     // years is not visible here any more; the following line
20     // would not compile:
21     // if (years >= 60) { ... } // error: 'years' was not declared
22
23     return 0;
24 }

```

样例输入 / 预期输出

```

1  25 =>  adult (7 year(s))
2  17 =>  adult in 1 year(s)

```

优势分析：

- **years 在 if 和 else 中都能使用**：初始化语句先执行，得到 `years`，然后判断 `years >= 0`；无论进入哪个分支，`years` 都可用，避免了重复计算；
- **years 不会泄漏到 if-else 之外**：出了 `if-else` 结构，`years` 立即不可见；如果后续代码试图使用 `years`，编译器会直接报错，把潜在的逻辑错误提前到编译期；
- **代码紧凑、意图清晰**：计算 `years` 和判断 `years >= 0` 紧挨在一起，读代码的人一眼就能看出：这个变量就是为这个判断服务的。



本示例符合 CG [ES.5: Keep scopes small](#)（保持作用域尽可能小）、[ES.20: Always initialize an object](#)（始终初始化对象）、与 [ES.23: Prefer the {}-initializer syntax](#)（优先使用 `{}` 统一初始化）。

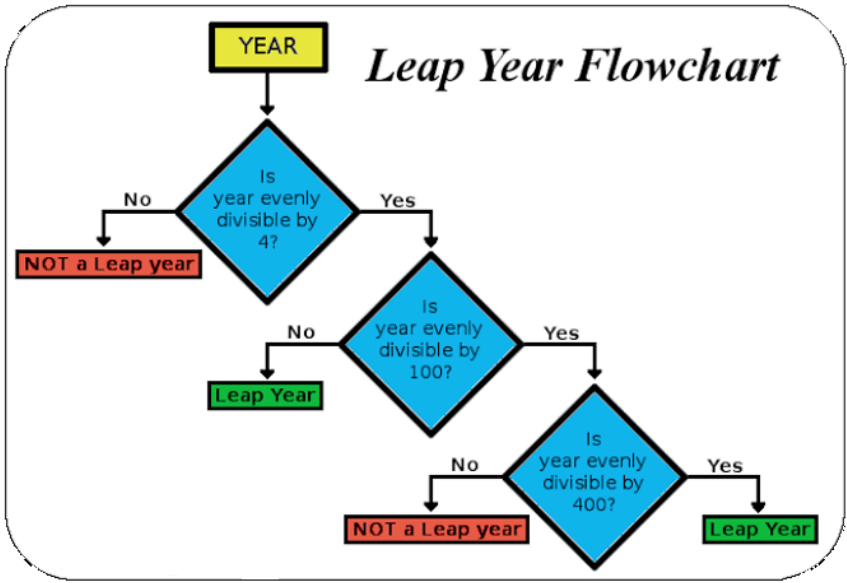
4.4 变量的作用域

写法	变量作用域	else 能否访问	是否泄漏到外部	代码重复	安全性
写法一：if 外部声明	整个函数	能	会	无	低，易误用
写法二：分支内声明	单个分支	不能	不会	有	中，但冗余
写法三：if 初始化器	整个 if-else	能	不会	无	高

if 初始化器声明的变量，作用域包括 if 和 else 分支，但不包括 if-else 之后的代码：变量生命周期与逻辑用途严格对齐，需要时存在、不需要时消失；如果忘记写初始化语句，语法退化为普通的 if，兼容旧代码。它适用于“需要先计算一个中间值，再根据这个值进行判断”的场景。

5. leap

判断给定的年份是否为闰年。



我们使用嵌套的 if-else 语句模拟上面的流程图（flowchart）。对于复杂的嵌套逻辑，绘制流程图可以辅助思考，避免逻辑错误。

5.1 leap-if-else

```
leap-if-else.cpp

1 // File: leap-if-else.cpp
2
3 #include <format>
4 #include <iostream>
5
```

```

6  int main() {
7      int year{};
8      std::cin >> year;
9
10     bool leap{false}; // boolean variable, indicator, flag
11
12     if (year % 4 == 0) {
13         if (year % 100 == 0) {
14             if (year % 400 == 0) {
15                 leap = true;
16             } else {
17                 leap = false;
18             }
19         } else {
20             leap = true;
21         }
22     } else {
23         leap = false;
24     }
25
26     if (leap) {
27         std::cout << std::format("{} is a leap year\n", year);
28     } else {
29         std::cout << std::format("{} is a common year\n", year);
30     }
31
32     return 0;
33 }

```

样例输入

```
1  2024
```

预期输出

```
1  2024 is a leap year
```

5.1.1 `bool leap{false}`

我们需要引入一个变量用来表征某个年份是否为闰年。从逻辑上讲，该变量只有两种可能的取值，要么为“真”（`true`）要么为“假”（`false`）。这种变量称为布尔（boolean）变量。C++ 原生提供了 `bool` 类型（参见 [bool @ cppreference](#)），其取值只有 `true` 与 `false` 两个关键字。

！ 注意：在条件判断中，`0` 表示“假”，非 `0` 表示“真”，这里的 `0` 不限于整数类型的数字 `0`，非 `0` 也不限于非零数字。更多内容，后续遇到时再作解释。

本例使用 `bool` 类型的变量 `leap` 表征某个年份是否为闰年：`true` 表示是闰年，`false` 表示不是闰年。我们也称这种变量为指示（indicator）变量或者标志（flag）变量。

📖 在 C 语言中，直到 C99 才引入 `_Bool` 类型（通过 `<stdbool.h>`），C23 才将 `bool`、`true`、`false` 提升为关键字；而 C++ 从一开始（C++98）就内置 `bool` 类型与 `true`、`false` 关键字。后续讲义（如 [§3-for-a-while](#)）中会大量使用 `bool` 变量。

5.1.2 `year % 4 == 0`

C++ 使用 `==` 判断相等性（equality）。初学者需要区分 `==` 与 `=`。`=` 表示赋值（assignment）。因此，从语义上讲，这里的 `if (year % 4 == 0)` 不能写成 `if (year % 4 = 0)`。

如果写成 `if (year % 4 = 0)`，则会导致编译错误。错误的原因是 `year % 4` 是一个右值（rvalue），无法取得它的地址（location），因此不能对其进行赋值。

📖 关于左值（lvalue）、右值（rvalue）以及与之紧密相关的对象（object）的概念，将在后续课程中详细介绍。此处不作要求。

关于 `==` 与 `=` 的区别，还有一个初学者更易犯错的地方。假设条件为 `if (year == 0)`，此条件成立当且仅当 `year` 的值为 0。如果错写成 `if (year = 0)`，编译器并不会报错！这是因为 `year = 0` 本身也是一个表达式，称为**赋值表达式**，经过求值（evaluation）后，其值就是 `year` 的值，即为 0。在 C++ 中，0 表示“假”（false），非 0 表示“真”（true）。因此，`if (year = 0)` 恒不成立！

📖 也正因为赋值表达式本身是有值的，C++ 允许将赋值串联起来，比如 `a = b = c = 1`。赋值操作符是右结合的，因此 `a = b = c = 1` 实际是 `(a = (b = (c = 1)))`，效果等同于 `a = 1; b = 1; c = 1`。

需要注意的是，赋值表达式的值是 `=` 左边的变量被赋值后的值，而不是 `=` 右侧的值。比如，`int a; double b; a = b = 1.5;` 的效果是 `a = 1; b = 1.5`（可自行在 [Compiler Explorer](#) 中验证）。

！ 有人建议将 `if (number == 1)` 写成 `if (1 == number)`，因为如果不慎误写成 `if (1 = number)`，则会导致编译错误，引起注意。

但我们不提倡这么做！首先，这会影响代码的可读性。其次，作为合格的程序员，你本就要训练自己不犯这种错误。最后，现代 IDE（如 CLion、VS Code）在你写出 `if (number = 1)` 时会给出警告，所以没有这么做的必要。

5.2 leap-else-if

leap-nested.cpp

```
1  // File: leap-nested.cpp
2  // The same logic as leap-elseif.cpp, but written with nested if-else
3  // instead of the else-if chain.
4
5  #include <format>
6  #include <iostream>
7
8  int main() {
9      int year{};
10     std::cin >> year;
11
12     bool leap{false};
13     if (year % 400 == 0) {
14         leap = true;
15     } else {
16         if (year % 100 == 0) {
17             leap = false;
18         } else {
19             if (year % 4 == 0) {
20                 leap = true;
21             } else {
22                 leap = false;
23             }
24         }
25     }
26
27     if (leap) {
28         std::cout << std::format("{} is a leap year\n", year);
29     } else {
30         std::cout << std::format("{} is a common year\n", year);
31     }
32
33     return 0;
34 }
```

样例输入

```
1 1900
```

预期输出

```
1 1900 is a common year
```

本段与 `leap-elseif.cpp` 的判断顺序相同（从特殊到一般：400 → 100 → 4），但把 `else` 后的 `if` 一层层嵌套起来。逻辑正确，但每多一层判断，代码就向右缩进一层，括号层级也随之加深；当分支较多时，这种写法难以阅读与维护。下一节用 `else if` 级联改写同一逻辑，两相对照即可看出后者的清晰与表达力。

5.3 `leap-elseif`

`leap-elseif.cpp`

```
1 // File: leap-elseif.cpp
2
3 #include <format>
4 #include <iostream>
5
6 int main() {
7     int year{};
8     std::cin >> year;
9
10    bool leap{false};
11    if (year % 400 == 0) {
12        leap = true;
13    } else if (year % 100 == 0) {
14        leap = false;
15    } else if (year % 4 == 0) {
16        leap = true;
17    } else {
18        leap = false;
19    }
20
21    if (leap) {
22        std::cout << std::format("{} is a leap year\n", year);
23    } else {
24        std::cout << std::format("{} is a common year\n", year);
25    }
26
27    return 0;
28 }
```

样例输入

```
1 1900
```

预期输出

```
1 1900 is a common year
```

`else if` 不是一个新的关键字，而是 `else` 后紧跟一个 `if` 语句的写法，用于表达互斥条件的链式判断。本段代码按“从特殊到一般”的顺序判断：能被 400 整除是闰年；否则能被 100 整除是平年；否则能被 4 整除是闰年；其余是平年。它与 5.2 节 `leap-nested` 的嵌套写法逻辑完全一致，但把 `else` 后的嵌套 `if` 平铺成 `else if` 链，层次更浅、更易读。

5.4 leap

leap.cpp

```
1 // File: leap.cpp
2
3 #include <format>
4 #include <iostream>
5
6 int main() {
7     int year{};
8     std::cin >> year;
9
10    bool leap{false};
11    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
12        leap = true;
13    }
14
15    // Equivalently, initialize leap directly:
16
17    if (leap) {
18        std::cout << std::format("{} is a leap year\n", year);
19    } else {
20        std::cout << std::format("{} is a common year\n", year);
21    }
22
23    return 0;
24 }
```

样例输入

```
1 2000
```

预期输出

```
1 2000 is a leap year
```

5.4.1 逻辑表达式

`(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)` 是由逻辑运算符 (logical operator) 与条件构成的逻辑表达式 (logical expression, 参见 [operator_logical @ cppreference](#))。逻辑运算符共三种, 分别是 `&&` (表示“并且”)、`||` (表示“或者”) 以及 `!` (表示“取反”)。`A && B` 为真当且仅当 `A`、`B` 均为真。`A || B` 为真当且仅当 `A`、`B` 至少一个为真。`!A` 为真当且仅当 `A` 为假。逻辑表达式的结果是 `bool` 类型的值: 为真 (`true`) 或为假 (`false`)。

三种逻辑运算符的优先级从高到低依次为 `!`、`&&`、`||`。不要依赖于它们的优先级, 建议多使用括号表示运算顺序。

! `&&` 不同于 `&`, `||` 不同于 `|`。`&` 与 `|` 表示位运算, 以后再作详细介绍 (软件学院《计算机系统基础》一课中也会有介绍)。

5.4.2 短路求值

C++ 标准规定, 按照短路求值 (short-circuit evaluation) 规则计算逻辑表达式的结果。具体而言:

- 对于 `A && B`, 先计算 `A` 的值, 如果 `A` 为假 (`false`), 则不再计算 `B` 的值 (`B` 被短路掉了), 整个逻辑表达式的结果为假 (`false`);
- 对于 `A || B`, 先计算 `A` 的值, 如果 `A` 为真 (`true`), 则不再计算 `B` 的值 (`B` 被短路掉了), 整个逻辑表达式的结果为真 (`true`)。

例如, 假设 `year` 的值是 25, 计算顺序如下:

- 先计算 `year % 4 == 0 && year % 100 != 0`
 - 先计算 `year % 4 == 0`, 其值为假
 - 跳过 `year % 100 != 0`
- 由于 `year % 4 == 0 && year % 100 != 0` 这一段已为假, 需要继续计算 `||` 右侧的 `year % 400 == 0`, 其值为假, 因此整个逻辑表达式的结果为假。

请自行思考 `year = 80`、`year = 100`、`year = 400` 时的计算顺序。

5.4.3 求值顺序

求值顺序 (order of evaluation) 指表达式中各个子表达式被计算的先后次序。C++ 标准只对少数运算符（如 `&&`、`||`、`? :`、逗号运算符）明确规定了求值顺序；对于大多数运算符，操作数的求值顺序是未指定的（unspecified）。因此，不要写出依赖求值顺序的代码。

典型错误是，在同一完整表达式中，对同一个对象的修改与对其值的读取之间没有确定的先后顺序。例如：

代码块

```
1  int x{i + i++};
```

`i++` 会修改 `i`，而 `+` 的两个操作数之间没有规定求值顺序。因此，左侧 `i` 的读取与右侧 `i++` 对 `i` 的修改是 *unsequenced*。根据 C++ 标准 `[intro.execution]`，若对某个内存位置的副作用与读取同一位置中对象值的 value computation 之间是 unsequenced，则行为未定义。



[Evaluation Order @ cppreference](#) 规定如下行为属于 UB: A side effect on a memory location is unsequenced relative to a value computation using the value of any object in the same memory location.

5.5 `[[likely]]` 与 `[[unlikely]]`

C++20 起，可在 `if` 语句的分支上标注属性 `[[likely]]` 与 `[[unlikely]]`，提示编译器某条执行路径更常见或更少见，从而可能影响基本块布局以及其他依赖分支概率的优化（参见 [\[\[likely\]\] @ cppreference](#)）。属性只表达程序员对分支概率的预期，不改变程序语义。

以 5.4 节 leap.cpp 的输出部分为例对比：

原代码（leap.cpp 的输出部分）

```
1  if (leap) {
2      std::cout << std::format("{} is a leap year\n", year);
3  } else {
4      std::cout << std::format("{} is a common year\n", year);
5  }
```

融合 `[[unlikely]]` / `[[likely]]`

```
1  if (leap) [[unlikely]] {
2      std::cout << std::format("{} is a leap year\n", year);
3  } else {
4      std::cout << std::format("{} is a common year\n", year);
5  }
```


平年比闰年更常见，因此可以将闰年分支标记为 `[[unlikely]]`，向优化器表达该执行路径相对较少出现。需要强调的是，该属性只是优化提示，并不保证生成不同的机器代码，也不保证实际性能提高。本例代码也仅是示例 `[[likely]]` 与 `[[unlikely]]` 的用法，并不代表可以获得性能提升。实际上，本程序的性能瓶颈在 `std::cout`，而不在闰年的判断上。

6. next-day

输入“年月日”，输出下一天的日期。

这是本节的 `if/else` 综合示例。它综合运用第 3、5 节的内容：`if-else` 链（计算当月天数）、逻辑表达式（闰年判断与日期合法性检查）、两个独立的 `if`（顺序推进月末、年末回绕）。尚未学习函数，全部逻辑写在 `main` 中：

next-day.cpp

```
1 // File: next-day.cpp
2 // Input a date (year month day), print the next day's date.
3 // Functions have not been introduced yet, so all logic is in main().
4
5 #include <format>
6 #include <iostream>
7
8 int main() {
9     int year{};
10    int month{};
11    int day{};
12    std::cin >> year >> month >> day;
13
14    // Days in the current month: an if-else chain
15    // (the same problem is solved with switch in Section 7.2)
16    bool leap{(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)};
17    int days{};
18    if (month == 2) {
19        days = leap ? 29 : 28;
20    } else if (month == 4 || month == 6 || month == 9 || month == 11) {
21        days = 30;
22    } else {
23        days = 31;
24    }
25
26    // Validate the date before advancing
27    if (month < 1 || month > 12 || day < 1 || day > days) {
28        std::cout << std::format("invalid date\n");
29        return 0;
30    }
31
```

```

32 // Advance to the next day: roll over the month first, then the year
33 day += 1;
34 if (day > days) {
35     day = 1;
36     month += 1;
37 }
38 if (month > 12) {
39     month = 1;
40     year += 1;
41 }
42
43 std::cout << std::format("{}-{}-{}\n", year, month, day);
44 return 0;
45 }

```

样例输入 / 预期输出

```

1  2024 2 28 => 2024-2-29
2  2023 2 28 => 2023-3-1
3  2024 4 30 => 2024-5-1
4  2024 12 31 => 2025-1-1
5  2024 1 1  => 2024-1-2
6  2024 6 15 => 2024-6-16
7  2024 2 30 => invalid date
8  2024 13 1 => invalid date

```

逐段讲解：

- Line 18 到 Line 24 用 if-else 链计算当月天数：2 月依赖闰年判断（复用 5 节的逻辑表达式，`?:` 来自 3.1.2 节）；4、6、9、11 月为 30 天；其余月份为 31 天。这一小段与 7.2 节 days-of-month 的 switch 版本计算的是同一问题，两相对照即可体会 if-else 与 switch 的取舍；
- Line 27 在推进前做日期合法性检查：`month` 必须在 1-12 之间，`day` 不能超过当月天数；非法输入（如 2024-2-30、2024-13-1）直接退出，不再计算；
- Line 33 到 Line 41 推进到下一天，用两个**独立的 if**（而非 else-if）：先处理“月末回绕”（`day` 超过当月天数则归 1、`month` 加一），再处理“年末回绕”（`month` 超过 12 则归 1、`year` 加一）。顺序是关键：先月末、后年末；
- 边界示例：2024-2-28 → 2024-2-29（闰年二月 29 天）、2024-12-31 → 2025-1-1（年末回绕到新年）。

7. switch/case

7.1 语法与语义

`switch` 语句用于“一组离散常量”的多分支选择：根据条件（condition）的值跳转到匹配的 `case` 分支执行（参见 [switch @ cppreference](#)）。语法要点：

- `switch` 的 condition 须为整型或枚举类型（不能是字符串、浮点等非整型/枚举类型）；
- `case` 标签必须是常量表达式（如 `case 2:`）；
- 每个 `case` 分支默认会“贯穿”（fall through）到下一个 `case`，通常以 `break` 结束，跳出整个 `switch` 语句；
- `default` 分支可选，处理“未列出的值”（惯例放在最后）。

7.2 days-of-month

输入年份与月份，输出该月天数。

一个有实际意义的 `switch/case` 例子。它与 5 节 leap 联动（二月天数依赖闰年判断），并示范 `case` 分组、`break` 与 `default`；输入读取融合了 `if` 语句带初始化器（详见 4 节）：

days-of-month.cpp

```
1  // File: days-of-month.cpp
2  // Print how many days a given month has.
3  // The input is read with an if statement with initializer (C++17):
4  // the scope of month is limited to this if statement.
5
6  #include <format>
7  #include <iostream>
8
9  int main() {
10     int year{};
11     int month{};
12
13     // We assume that 1 <= month <= 12
14     std::cin >> year >> month;
15
16     bool leap{(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)};
17
18     int days{};
19     switch (month) {
20         case 2:
21             days = leap ? 29 : 28;
22             break;
23         case 4:
24         case 6:
25         case 9:
26         case 11:
27             days = 30;
28             break;
```

```

29     default:
30         days = 31;
31         break;
32     }
33     std::cout << std::format("{} / {} has {} days\n", year, month, days);
34
35     return 0;
36 }

```

样例输入 / 预期输出

```

1  2024 2 => 2024/2 has 29 days
2  2023 2 => 2023/2 has 28 days
3  2024 4 => 2024/4 has 30 days
4  2024 1 => 2024/1 has 31 days

```

教学点：

- Line 19 到 Line 32 是 `switch` 语句： `case 4`、`case 6`、`case 9`、`case 11` 连续写成四个 `case` 共享同一分支（空的 `case` 贯穿合法且常见）； `default` 处理其余月份；
- 二月分支依赖闰年判断： `days = leap ? 29 : 28;` 复用 5 节 `leap` 的逻辑表达式与 3.1.2 节的 `?:`；
- 贯穿大多数时候是 bug（漏写 `break`），编译器以 `-Wimplicit-fallthrough` 警告；确为有意时用 `[[fallthrough]]` 显式声明，既消除警告，又让意图对读者可见。

代码块

```

1  switch (month) {
2      case 2:
3          bool leap{
4              (year % 4 == 0 && year % 100 != 0) ||
5              (year % 400 == 0)
6          };
7
8          days = leap ? 29 : 28;
9          break;
10
11     case 4:
12
13 }

```

所有 `case` 共享同一个作用域。在 `case` 中直接声明变量，可能因 `switch` 的跳转而跳过其初始化（例如直接跳到声明处之后的语句），因此需要用花括号把变量限定在所在 `case` 内。

days-of-month.cpp

```
1  // File: days-of-month.cpp
2  // Print how many days a given month has.
3  // Demonstrates fall-through and block scope in a switch statement.
4
5  #include <format>
6  #include <iostream>
7
8  int main() {
9      int year{};
10     int month{};
11
12     // We assume that 1 <= month <= 12.
13     std::cin >> year >> month;
14
15     int days{};
16
17     switch (month) {
18         case 2: {
19             bool leap{
20                 (year % 4 == 0 && year % 100 != 0) ||
21                 (year % 400 == 0)
22             };
23
24             days = leap ? 29 : 28;
25             break;
26         }
27
28         case 4:
29         case 6:
30         case 9:
31         case 11:
32             days = 30;
33             break;
34
35         default:
36             days = 31;
37             break;
38     }
39
40     std::cout << std::format(
41         "{}/{ } has { } days\n", year, month, days);
```

```
42
43     return 0;
44 }
```



本示例符合 CG [ES.5: Keep scopes small](#)（保持作用域尽可能小）。



C++ Core Guidelines 的 [ES.22: When you need a variable, prefer to declare it at its first use](#) 建议：直到有值可初始化、且确需使用时才声明变量

7.3 temperature-control

温室中的数字温度传感器存在固定测量偏差。系统先读取传感器的原始温度

`raw_temperature`，再加上校准值 `calibration`，得到校准后的温度 `temperature`。温控系统不针对每一个具体温度采取措施，而是按温度区间分类：低温开加热器、正常、偏高开通风、高温报警。假设校准后的温度在 0°C 到 99°C 之间。

这个例子同时示范三件事：把原始读数**校准**为真正想用的温度；把连续的温度值**映射**为若干区间供 `switch` 分类；以及 `switch` 带初始化器（C++17）如何把中间变量 `temperature` 的作用域限定在 `switch` 之内。

temperature-control.cpp

```
1  // File: temperature-control.cpp
2  // A greenhouse temperature controller.
3  // Read the raw sensor reading, apply the calibration offset, and
4  // choose a control action by the calibrated temperature band.
5  // switch with an initializer (C++17) keeps temperature local to it.
6
7  #include <format>
8  #include <iostream>
9
10 int main() {
11     int raw_temperature{};
12     int calibration{};
13     std::cin >> raw_temperature >> calibration;
14
15     // Assume 0 <= temperature <= 99.
16     switch (int temperature{raw_temperature + calibration};
17             temperature / 10) {
18     case 0:
19     case 1:
20         std::cout << std::format(
21             "Temperature: {} C -- turn on the heater.\n", temperature);
```

```

22     break;
23     case 2:
24     case 3:
25         std::cout << std::format(
26             "Temperature: {} C -- temperature is normal.\n", temperature);
27         break;
28     case 4:
29     case 5:
30         std::cout << std::format(
31             "Temperature: {} C -- turn on ventilation.\n", temperature);
32         break;
33     default:
34         std::cout << std::format(
35             "Temperature: {} C -- warning: too hot!\n", temperature);
36         break;
37 }
38
39 // temperature is out of scope here.
40
41 return 0;
42 }

```

样例输入 / 预期输出

```

1  35 -1 => Temperature: 34 C -- temperature is normal.
2  5 2  => Temperature: 7 C -- turn on the heater.
3  48 0  => Temperature: 48 C -- turn on ventilation.
4  80 5  => Temperature: 85 C -- warning: too hot!

```

switch 带初始化器：temperature 的作用域

C++17 起，`switch` 语句可以带初始化分句：`switch (init-statement; condition)`，与 4 节的 `if` 初始化器同构。朴素写法把 `temperature` 声明在 `switch` 外面：

朴素写法（temperature 泄漏到外层作用域）

```

1  int temperature{raw_temperature + calibration};
2  switch (temperature / 10) {
3      // ...
4  }
5  // temperature is still in scope here.

```

`temperature` 只用于这一次区间分类，却留在外层作用域，容易被后续代码误用。把声明挪进初始化分句，变量的作用域恰好覆盖整个 `switch`：

带初始化器（temperature 出 switch 即不可见）

```
1  switch (int temperature{raw_temperature + calibration};
2          temperature / 10) {
3      // ...
4  }
5  // temperature is out of scope here.
```

7.4 if-else vs. switch-case

当分支是“一组离散的常量”（如月份、会员等级）时，倾向于使用 `switch`，它结构清晰、可读性好，也便于编译器生成跳转表优化。

 C++ Core Guidelines 的 [ES.70: Prefer a switch-statement to an if-statement when there is a choice](#) 建议：有选择时，优先使用 `switch` 而非 if-else 链。

8. 修订记录

- 2026-09-17：统一代码块 caption 格式——去掉文件名后的（C++20）标记（如 `.cpp`（C++20）→ `.cpp`）。
- 2026-09-19：7.3 节整合温控系统示例 `temperature-control.cpp`——示范 `switch` 带初始化器（C++17）、`case` 分组与连续温度区间映射（`/10`）。
- 2026-09-19：按 1-types-io 与《AI 辅助写作准则》审阅修订——修复 `leap.cpp` 注释乱码、短路求值讲解逻辑、行号引用（`min-of-two` / `next-day` / `days-of-month`）。

（注：部分内容可能由 AI 生成）